

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота №2

Реалізація схеми обміну даними між програмним забезпеченням
(SW) та апаратною частиною (HW) з використанням інтерфейсу
UART

Виконав: ст. гр. КІ-404

Шевчук О.В.

Прийняв:

Федак П.Р.

Львів 2024

Мета роботи: Розробити просту систему комунікації між програмним клієнтом та апаратним сервером, використовуючи UART для обміну повідомленнями. Перевірити коректність обміну даними, включаючи відправлення, модифікацію та зворотну передачу повідомлень. Налаштувати автоматизацію збірки, тестування та генерації артефактів за допомогою YML-файлу.

ТЕОРЕТИЧНІ ВІДОМОСТІ

UART (Universal Asynchronous Receiver-Transmitter) — це асинхронний послідовний інтерфейс для передачі та прийому даних між пристроями. Він використовується для обміну інформацією між комп'ютерами та периферійними пристроями, такими як мікроконтролери, датчики, модеми тощо. Основні характеристики UART:

- **Послідовна передача даних:** дані передаються по одному біту за раз через лінію передачі.
- **Швидкість передачі даних (baud rate):** кількість бітів, що передаються за секунду. Відправник і приймач повинні бути налаштовані на однакову швидкість.
- **Формат кадру:** складається зі стартового біта, даних (від 5 до 8 біт), необов'язкового парного біта для перевірки помилок і стопового біта.

Клієнт-серверна архітектура передбачає наявність двох основних компонентів: клієнта та сервера.

- **Клієнт:** ініціює запит на передачу даних до сервера.
- **Сервер:** приймає дані, виконує над ними певні дії (наприклад, модифікацію) і надсилає змінені дані назад клієнту.

У даній роботі клієнт відповідає за відправлення повідомлення до сервера через UART, сервер обробляє це повідомлення і повертає результат назад клієнту.

YML (YAML Ain't Markup Language) — формат файлу, який часто

використовується для конфігурацій та автоматизації процесів в системах безперервної інтеграції та розгортання (CI/CD). У цьому проєкті YML-файл виконує такі завдання:

- **Збірка проєкту:** автоматична компіляція та створення бінарних файлів.
- **Тестування:** запуск тестів для перевірки функціональності програмного забезпечення.
- **Генерація артефактів:** створення звітів та інших файлів для подальшого використання або аналізу.

ЗАВДАННЯ

1. Розробити схему обміну даними між програмним забезпеченням (SW) та апаратною частиною (HW), використовуючи інтерфейс UART.
2. Реалізувати програмний клієнт, який надсилає повідомлення до сервера.
3. Реалізувати серверну частину, що приймає повідомлення, змінює його вміст та надсилає модифіковане повідомлення назад клієнту.
4. Перевірити коректність роботи схеми, зокрема:
 - правильність передачі даних від клієнта до сервера;
 - обробку даних на сервері (модифікацію повідомлення);
 - зворотну передачу модифікованого повідомлення до клієнта.
5. Створити YML-файл для автоматизації таких процесів:
 - збірка всіх бінарних файлів проєкту;
 - запуск тестів для перевірки функціональності;
 - створення артефактів, що містять бінарні файли та результати тестування.
6. Налаштувати та розробити скрипти для автоматизації процесів у каталозі `ci/`.
7. Оформити звіт, в якому надати результати тестування, вихідний код реалізації та опис процесу виконання роботи.

ХІД РОБОТИ

Оскільки для комунікації SW і HW по UART я буду використовувати сервер симулятор мікроконтролера Arduino UNO Wokwi. Мені знадобиться програма симуляції віртуальних COM портів. Тому я завантажив HHD virtual serial port. Та увімкнув комунікацію між COM1 і COM2.

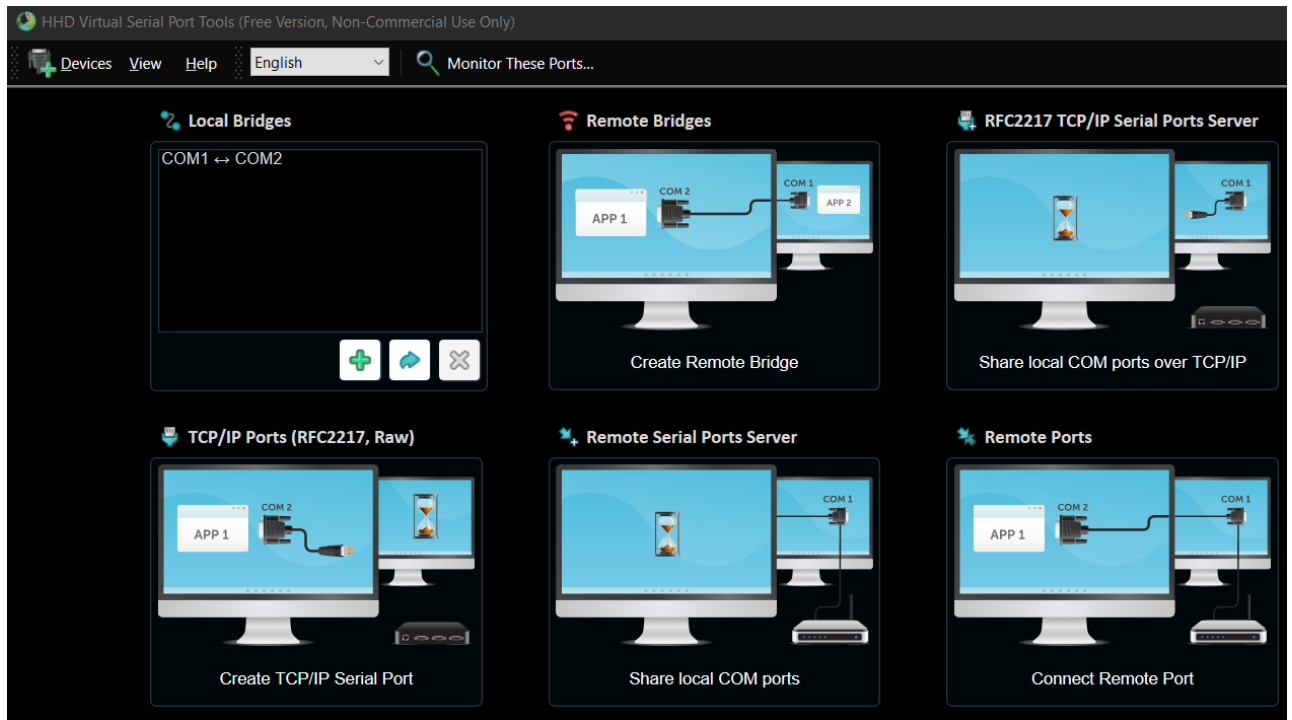


Рис.1. Налаштування віртуальних COM портів

Перевірив правильність налаштування віртуальних COM портів.

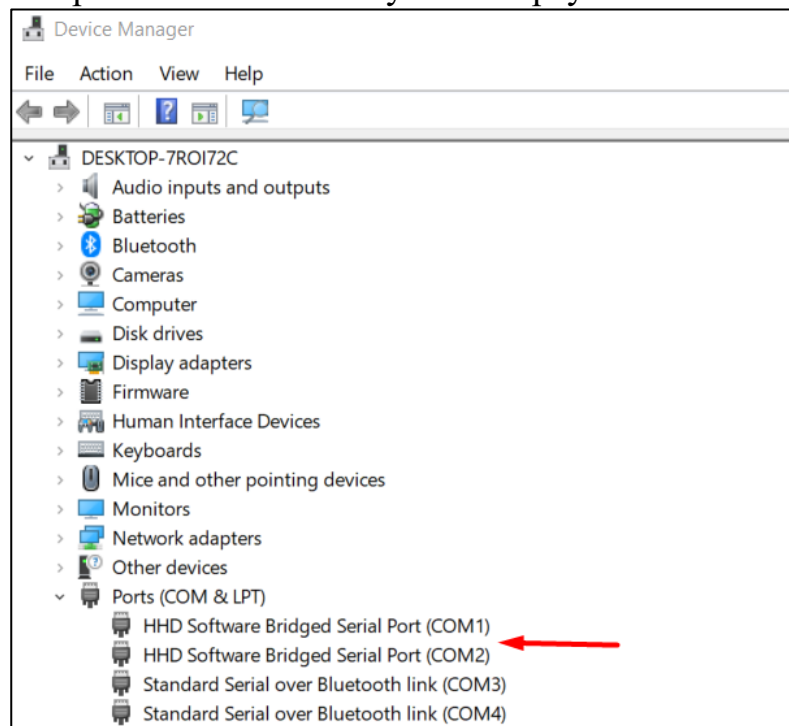


Рис.2. Наявність COM портів

Написав код для клієнта мовою програмування C# з використанням бібліотеки System.IO.Ports, що використовується для роботи з портами, в нашому випадку COM порти.

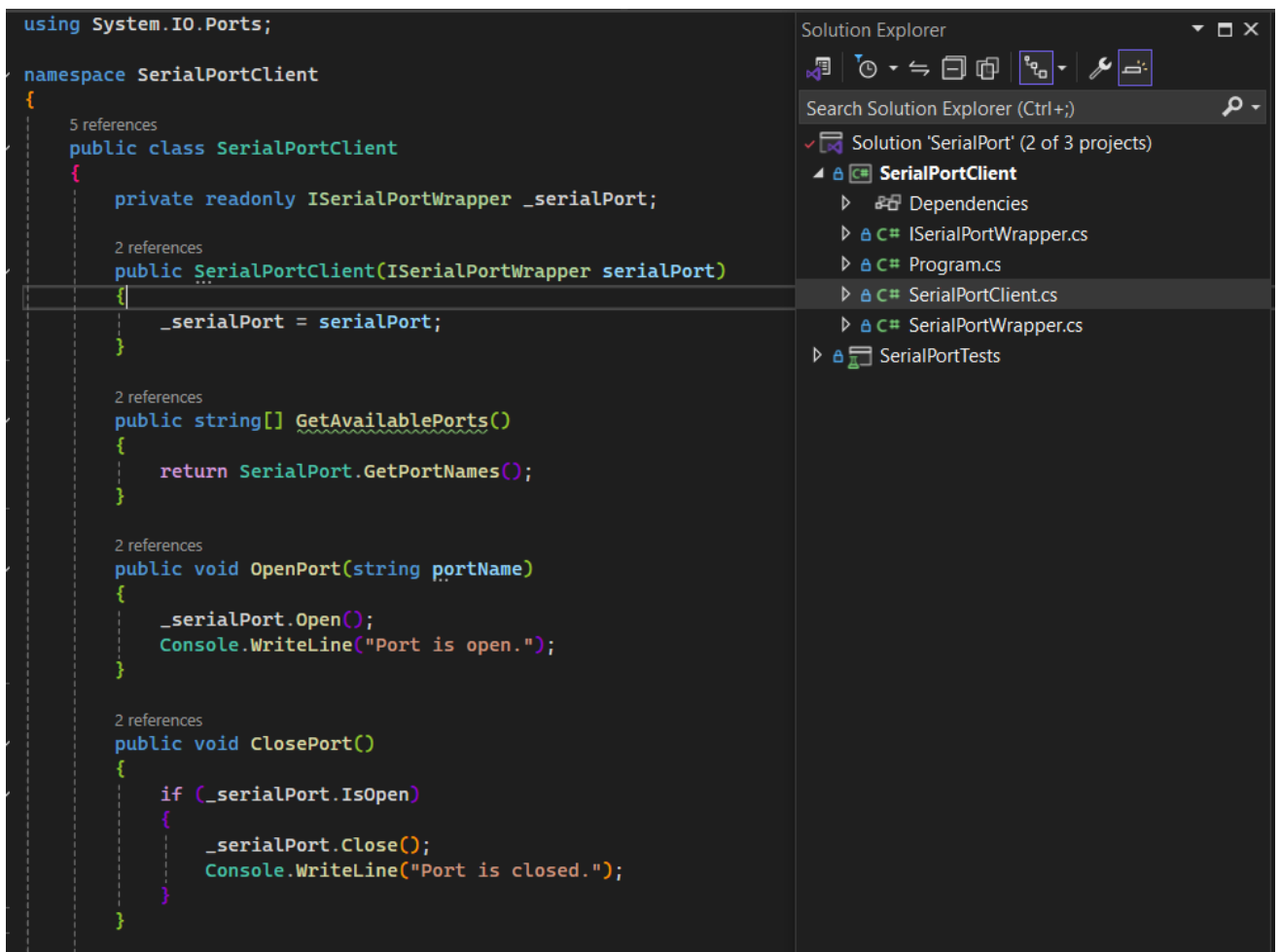


Рис.3. Створення SW частини

Написав код для серверу в симуляторі Wokwi.

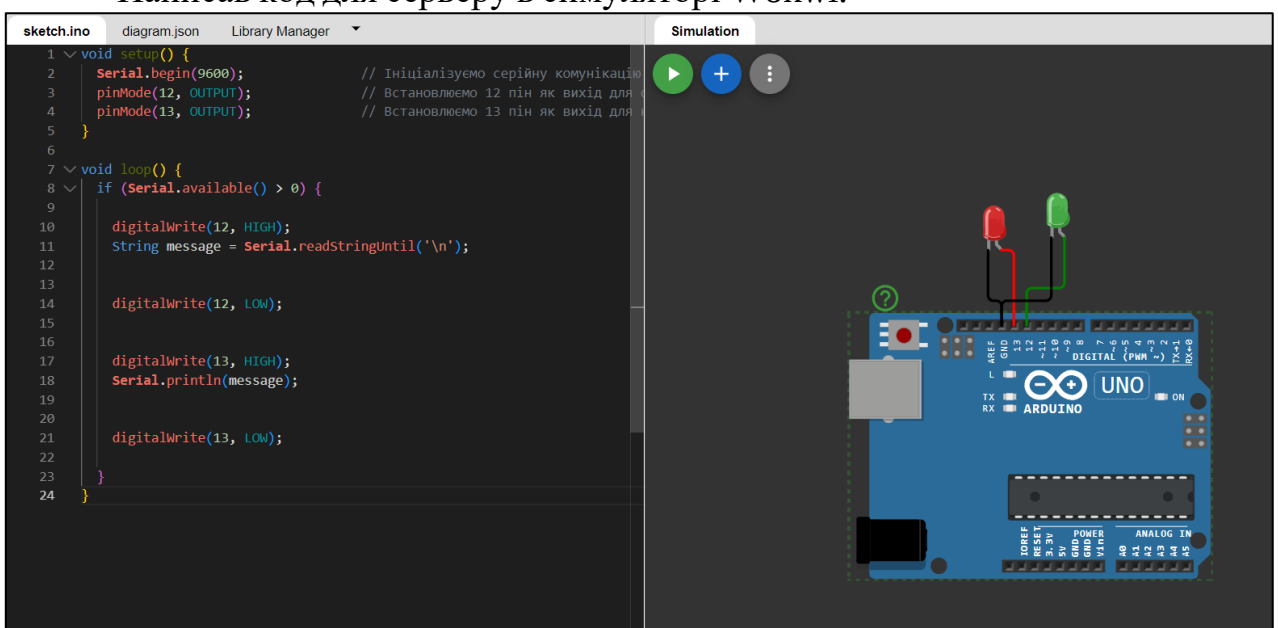


Рис.4. Створення HW частини

Перевіряю комунікацію між клієнтом і сервером. Запускаю сервер і клієнт і відповідно для кожного обираю COM1 і COM2.

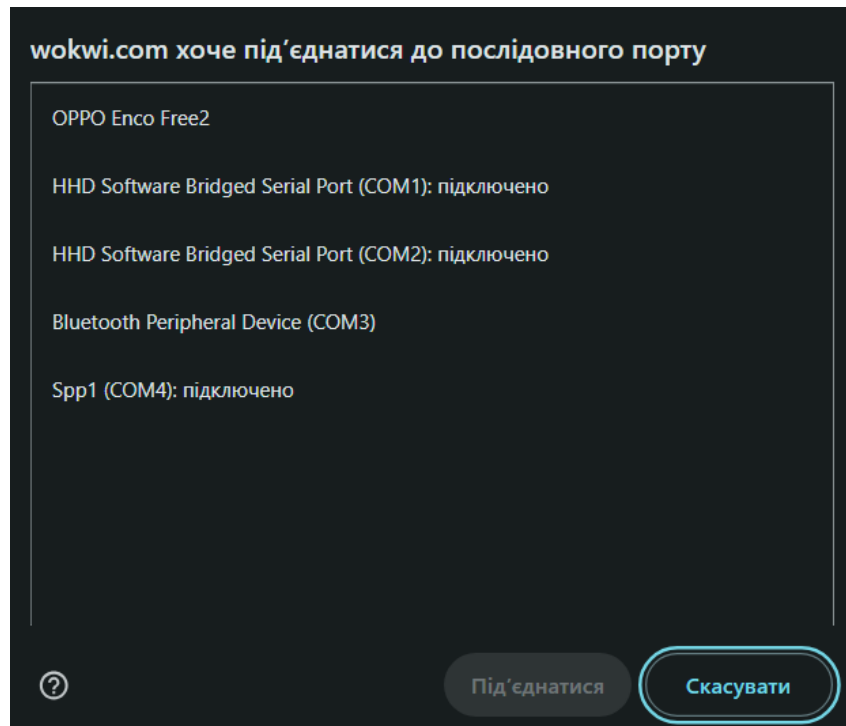


Рис.5. Обираю COM порт для серверу

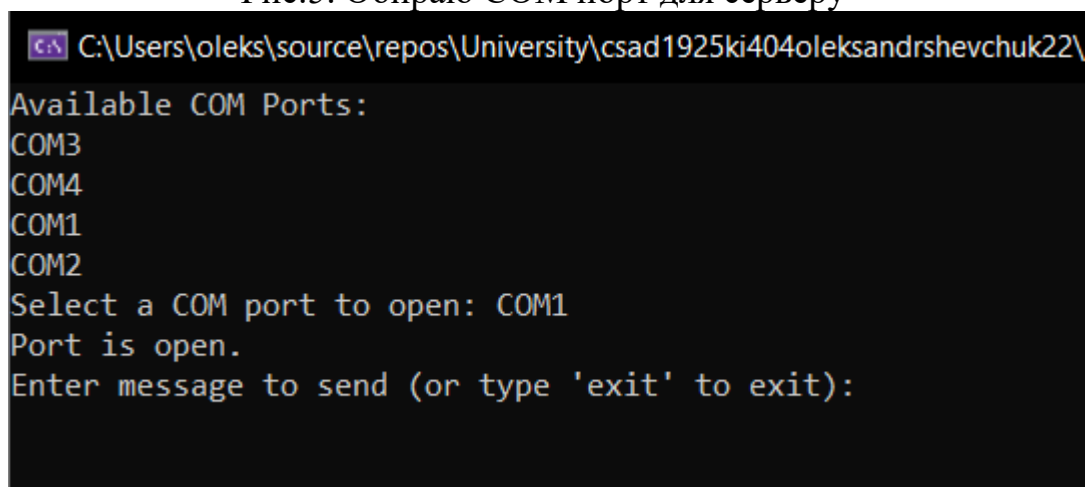


Рис.6. Обираю COM порт для клієнта

Перевіряю чи успішно створив комунікацію між клієнтом і сервером. Надсилаю повідомлення на сервер і очікую відповідь.

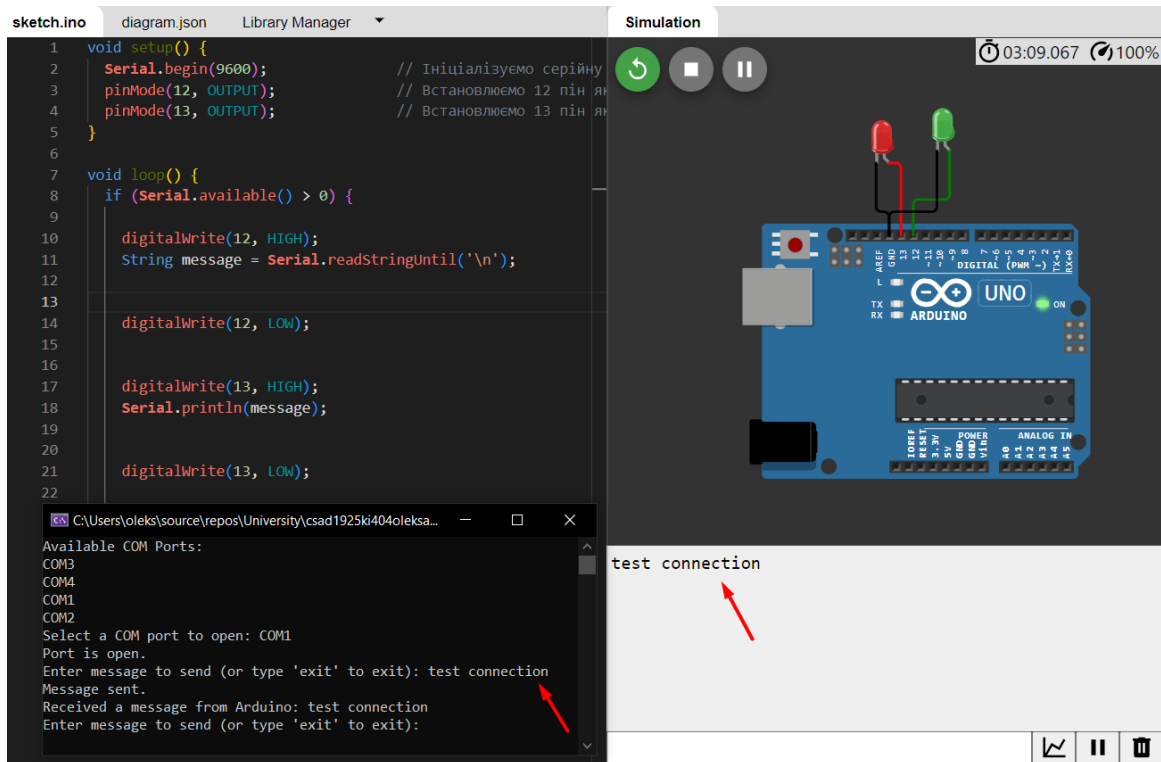


Рис.6. Комунікація. Між HW і SW

Написав тести для методів класу SerialPortClient

```

namespace SerialPortClient.Tests
{
    public class SerialPortClientTests
    {
        private readonly SerialPortClient _client;
        private readonly Mock<ISerialPortWrapper> _serialPortMock;

        public SerialPortClientTests()
        {
            // Mock the ISerialPortWrapper interface
            _serialPortMock = new Mock<ISerialPortWrapper>();
            _client = new SerialPortClient(_serialPortMock.Object);
        }

        [Fact]
        public void GetAvailablePorts_ShouldReturnAvailablePorts()
        {
            // Act
            var ports = _client.GetAvailablePorts();

            // Assert
            Assert.NotNull(ports);
        }

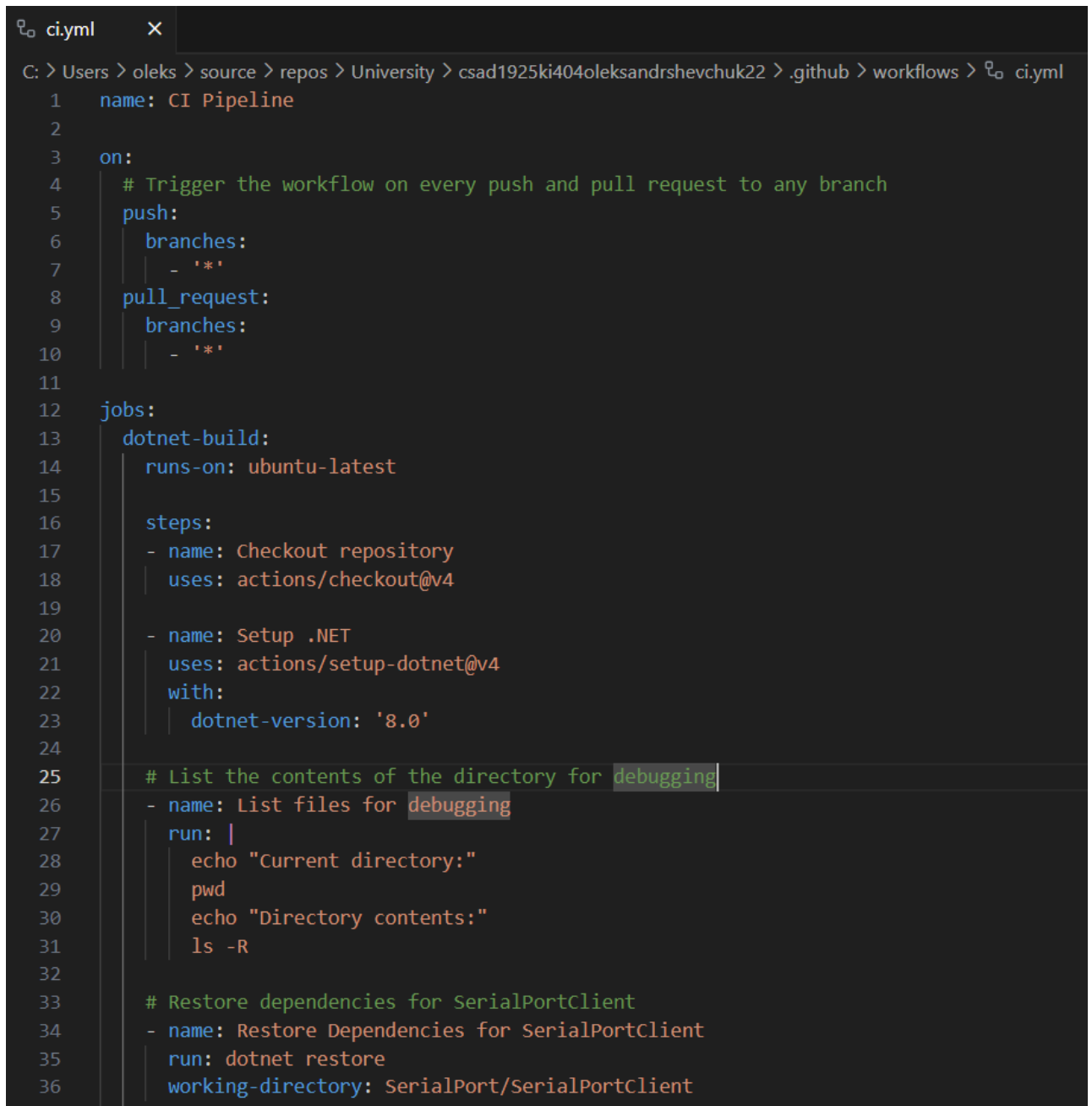
        [Fact]
        0 references
        public void OpenPort_ShouldOpenSerialPort()
        {
            // Arrange
            _serialPortMock.Setup(sp => sp.Open());

            // Act
            _client.OpenPort("COM1");
        }
    }
}

```

Рис.7. Тести для SW

Далі створив ci.yml і написав код для CI, щоб автоматизувати процеси тестування і розгортання проекту.



```
1  name: CI Pipeline
2
3  on:
4    # Trigger the workflow on every push and pull request to any branch
5    push:
6      branches:
7        - '*'
8    pull_request:
9      branches:
10       - '*'
11
12  jobs:
13    dotnet-build:
14      runs-on: ubuntu-latest
15
16      steps:
17        - name: Checkout repository
18          uses: actions/checkout@v4
19
20        - name: Setup .NET
21          uses: actions/setup-dotnet@v4
22          with:
23            dotnet-version: '8.0'
24
25        # List the contents of the directory for debugging
26        - name: List files for debugging
27          run: |
28            echo "Current directory:"
29            pwd
30            echo "Directory contents:"
31            ls -R
32
33        # Restore dependencies for SerialPortClient
34        - name: Restore Dependencies for SerialPortClient
35          run: dotnet restore
36          working-directory: SerialPort/SerialPortClient
```

Рис.7. Створення CI

Завантажив проект на github і перевірів процес збірки. Як бачимо процес збірки відбувся успішно.

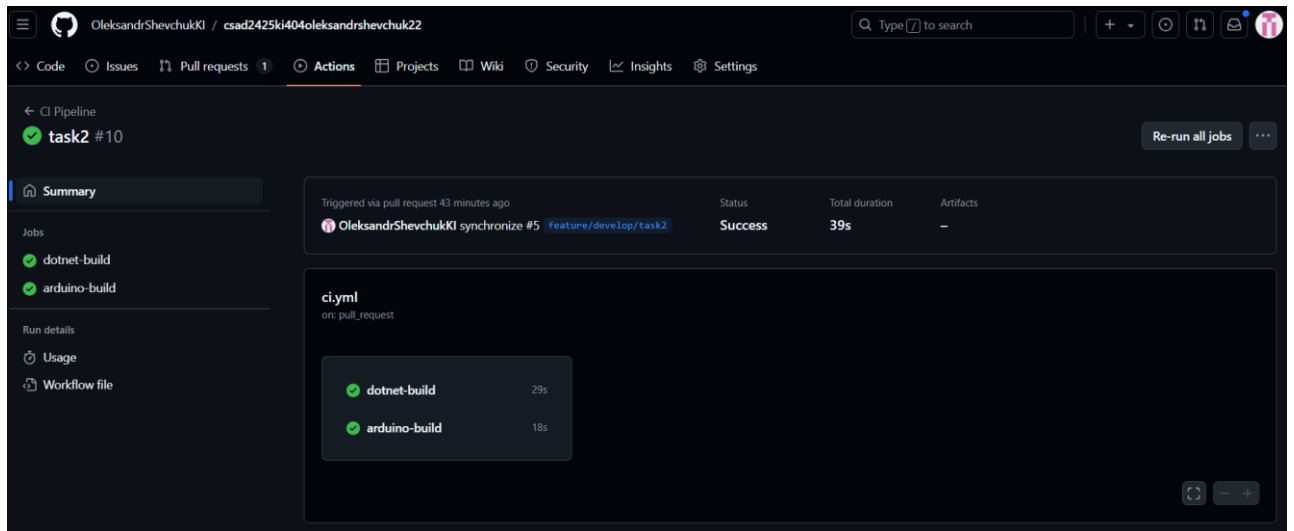


Рис.7. Успішне виконання CI

Висновок: Під час виконання роботи була реалізована система комунікації між програмним забезпеченням і апаратною частиною через UART. Успішно налаштовано передачу даних від клієнта до сервера, модифікацію повідомлень на сервері та зворотну передачу модифікованих даних до клієнта. Також було створено YML-файл для автоматизації збірки, тестування та створення артефактів, що дозволило оптимізувати процес розробки та забезпечити високий рівень контролю якості..